OWL 온톨로지 실습

owlready2를 활용한 제조 지식 표현 · 추론 · 질의

Day4_1 OWL 기본 구조
Day4_2 규칙 기반 추론
Day4_3 SWRL 규칙
Day4_4 SPARQL 질의
Day4_5 심화 분석
Day4_6 LangChain 연동

Day4_1 OWL 기본 구조 — 클래스 & 속성 정의

day4_1_owl_basic.py

목적: owlready2로 제조 도메인 온톨로지를 코드로 직접 정의 — 클래스·속성·개체·관계를 한 파일에서 완성

정의한 클래스 (6개)

- 제품 — 완성품 (예: 산업용펌프, 압력펌프)
- 부품 — 구성 요소 (예: 모터, 임펠러)
- 대형부품 — 부품의 하위 클래스 (상속 관계)
- 공정 — 작업 단계 (조립/가공/검사)
- 설비 — 기계·장비 (선반기계, 용접기 등)
- 작업자 — 공정 담당자

정의한 속성

- [Object] 포함한다 (제품 → 부품)
- [Object] 사용한다 (공정 → 설비)
- [Object] 담당한다 (공정 → 작업자)
- [Data] 무게 (부품, float)
- [Data] 작업시간 (공정, int)
- [Data] 불량율 (부품, float)

Day4_1 개체(Individual) 생성 & 관계 설정

day4_1_owl_basic.py

```
# 개체 생성 + 속성값 부여
모터    = 부품("모터");    모터.무게    = [5.0]
임펠러 = 부품("임펠러"); 임펠러.무게 = [2.0]
베어링 = 부품("베어링"); 베어링.불량율 = [0.05]

# 관계 설정 (포함한다)
산업용펌프.포함한다 = [모터, 임펠러, 케이싱]
모터.포함한다        = [베어링, 샤프트]

# 공정-설비 연결
조립공정1.사용한다 = [선반기계A]
```

BOM 계층 구조

산업용펌프

```
├ 모터 (5.0kg)
│    ├ 베어링 (0.5kg)
│    └ 샤프트 (1.5kg)
├ 임펠러 (2.0kg)
└ 케이싱 (8.0kg)
```

day4_1_owl_basic.py

1

with onto: 블록

모든 클래스·속성·개체 정의는 with onto: 블록 안에서 작성해야 온톨로지에 등록됨

2

클래스 상속

class 대형부품(부품): 처럼 파이썬 클래스 상속 문법으로 is-a 관계 표현

3

AllDisjoint

서로 다른 개념(제품, 공정, 설비)은 AllDisjoint로 명시 — 추론 오류 방지

4

ObjectProperty vs DataProperty

개체 간 관계 = ObjectProperty / 숫자·문자 값 = DataProperty

5

속성값은 리스트

b.무게 = [5.0] 처럼 리스트로 대입 → 조회 시 b.무게[0] 으로 접근

Day4_2 규칙 기반 추론 — 3가지 규칙 적용

day4_2_owl_reasoning.py

목적: 저장된 .owl 파일을 로드하여 데이터 기반 추론(분류)을 Python 코드로 구현

규칙 1

조건: 무게 ≥ 5kg

→ 대형부품

규칙 2

조건: 불량율 ≥ 3%

→ 불량위험부품

규칙 3

조건: 작업시간 ≥ 30분

→ 장시간공정

```
# 규칙 1 구현 예시
for b in onto.부품.instances():
    if b.무게 and b.무게[0] >= 5.0:
        대형부품_목록.append(b)

# 규칙 2 구현 예시
    if b.불량율 and b.불량율[0] >= 0.03:
        불량위험_목록.append(b)
```

추론 결과

- 대형부품: 모터(5.0kg), 케이싱(8.0kg)
- 불량위험: 베어링(불량율 5%)
- 장시간: 조립공정1(30분), 가공공정1(60분)

Day4_2 불량 부품 역추적 (Backward Tracing)

day4_2_owl_reasoning.py

핵심: 불량 부품이 어느 제품에 포함되어 있는지 관계를 거슬러 올라가며 탐색

```
# 역추적 로직
for 불량부품 in 불량위험_목록:
    for 제품 in onto.제품.instances():
        # 직접 포함 관계 확인
        if 불량부품 in 제품.포함한다:
            print(f"{제품.name} → 직접포함 → {불량부품.name}")
        # 2단계 포함 관계 확인
        for 중간부품 in 제품.포함한다:
            if hasattr(중간부품, '포함한다') and \
               불량부품 in 중간부품.포함한다:
                print(f"  {제품.name}→{중간부품.name}→{불량부품
.name}")
```

역추적 결과

불량부품: 베어링 (5%)

산업용펌프

→ 모터

→ 베어링

2단계 관계로 영향받음

포인트: 온톨로지의 관계(포함한다)를 활용하면 영향 범위를 자동으로 추적 가능

Day4_3 SWRL 규칙 — 복합 조건 추론

day4_3_swrl_rules.py

목적: 여러 조건을 AND로 결합한 복합 추론 규칙을 적용해 고위험부품·핵심부품·긴급점검공정을 자동 분류

규칙 1

조건: 불량율 > 3%

→ 고위험부품
+ 점검필요=True

규칙 2

조건: 2개 이상 제품에 사용됨

→ 핵심부품
+ 우선순위=1

규칙 3

조건: 작업시간 > 45분

→ 긴급점검공정

규칙 4

조건: 핵심부품
+ 불량위험

→ 즉시점검!
우선순위=0

Day4_3 구현 코드 & 추론 결과

day4_3_swrl_rules.py

```
# 새 클래스 추가 (with onto 블록 안)
class 고위험부품(onto.부품): pass
class 핵심부품(onto.부품): pass

# 새 속성 추가
class 점검필요(DataProperty):
    domain = [onto.부품]; range = [bool]
class 우선순위(DataProperty):
    domain = [onto.부품]; range = [int]

# 규칙 1 적용
for b in onto.부품.instances():
    if b.불량율 and b.불량율[0] > 0.03:
        b.is_a.append(onto.고위험부품)
        b.점검필요 = [True]
        b.우선순위 = [1]

# 규칙 4: 핵심 + 불량 → 즉시점검(우선순위 0)
if is_핵심 and is_불량위험:
    b.우선순위 = [0]
```

최종 점검 우선순위 출력

- 우선순위 0: 베어링 [고위험부품, 핵심부품] 즉시점검
- 우선순위 1: 케이싱 [대형부품]
- 우선순위 1: 모터 [대형부품]

b.is_a.append(클래스)

개체에 클래스를 동적으로 추가하는 owlready2 핵심 패턴

결과 파일: manufacturing_swrl.owl 저장 → Day4_4, Day4_5에서 이 파일을 계속 사용

Day4_4 SPARQL 질의 — 온톨로지 데이터 검색

day4_4_owl_sparql.py

목적: SPARQL(그래프 DB 질의 언어)로 온톨로지에 저장된 정보를 SQL처럼 조회

SPARQL 쿼리 구조

```
PREFIX  : <네임스페이스 URI>
SELECT  ?변수1 ?변수2 ...
WHERE {
    ?변수 a :클래스 .
    ?변수 :속성 ?값 .
}
ORDER BY  ?정렬기준
```

4가지 조회 쿼리

- ① 모든 부품과 무게 조회 (ORDER BY DESC 무게)
- ② 고위험부품 조회 (a :고위험부품)
- ③ 제품-부품 관계 조회 (:포함한다)
- ④ 점검필요 부품 조회 (:점검필요 true)

Day4_4 SPARQL 코드 & 결과

day4_4_owl_sparql.py

```
# owlready2 SPARQL 실행 방법
result = list(default_world.sparql("""
    PREFIX : <http://manufacturing.org/ontology#>
    SELECT ?부품 ?무게
    WHERE {
        ?부품 a :부품 .
        ?부품 :무게 ?무게 .
    }
    ORDER BY DESC(?무게)
"""))

# 결과 처리 (네임스페이스 제거)
for row in result:
    이름 = str(row[0]).split('#')[-1]
    print(f"{이름}: {row[1]}kg")
```

무게 내림차순 결과

케이싱	**8.0kg**
모터	**5.0kg**
임펠러	**2.0kg**
샤프트	**1.5kg**
베어링	**0.5kg**

포인트: str(row[0]).split('#')[-1] — URI에서 개체 이름만 추출하는 패턴을 반드시 기억